

Triron 1.1: a multi-axis substrate for resource-bounded continual learning

Marcelinus R. Hatorangan*

May 24, 2026

Abstract

We describe *Triron 1.1*, a substrate-level revision to the triron architecture that factors the monolithic v1.0 node into six orthogonal per-cell axes of structural plasticity: long-range input reach, plastic fanout sparsity, depth growth, slow axonal gain, dendritic compartmentalization, and field-conditional layer emergence. Each axis is a separate write channel into the cell’s state, opt-in and default-off, so that a v1.1 substrate with all axes at their defaults reproduces v1.0 behavior byte-identically. A spatial primitive—continuous positions on a unit square, partitioned into pools—then enables *pool-matched absorption*, a cell-granularity composition operator: two organisms trained independently on disjoint task slices can be merged into one substrate without retraining, provided they share the L0 perception layer’s deterministic factor (the $R \cdot S$ handshake). First verification on a chained-15 disjoint-class split shows task-aware accuracy preserved within $\Delta \leq 0.026$ on both donors, full-softmax accuracy lifted +0.20 to +0.31 via manifold merge plus 200-step head calibration, and post-absorption locality 0.78+ (vs. donor-B solo 0.58). On the same benchmark, the v1.1 credit-assignment plus manifold-replay recipe reaches 0.959 ± 0.001 task-aware and 0.563 ± 0.004 full-softmax at $n=3$ seeds and 8 epochs, comparable to the v1.0 headline (0.961 task-aware, 0.601 full at $n=10$). Axes 3, 4, and 5 regress when activated jointly on the main benchmark; Axis 3 (depth growth) is now default-off, and we report these regressions transparently. The 1.1 substrate is backwards-compatible with 1.0 donors: legacy checkpoints load unchanged and run under the extended forward path.

1 Introduction

This paper is a direct continuation of our earlier work on the triron architecture (Hatorangan, 2026), which introduced a continual-learning substrate built on three per-cell scalars—weight w , EWC lock λ , and utility u —plus aggregate machinery (frustration-triggered growth, dream-cycle consolidation, manifold replay). That v1.0 architecture, described in the companion paper (currently unpublished), was monolithic: every cell read its predecessor layer’s full output, broadcast a single scalar downstream, lived at a fixed depth, had a flat dense fan-in, and emerged through a uniform spawn rule. The resulting substrate learned effectively but offered no compositional surface—one could not selectively enable skip connections, locality, or depth growth without surgery on the forward path.

Triron 1.1 unpacks each of those implicit properties into a separate, opt-in *axis*. Each axis has its own write API, its own plasticity gate, and its own default (always the v1.0 behavior),

*Correspondence: marcelinusrocky@aol.com. Repository: <https://github.com/Marcrockhat/triron-project>. Live demo: huggingface.co/spaces/Marcrockhat/triron-demo. Guided tour: marcrockhat.github.io/triron-project/tour.

so that a default-constructed v1.1 cell is byte-identical to v1.0. The six axes are: (1) long-range input reach, (2) plastic fanout sparsity, (3) depth growth, (4) slow axonal gain, (5) dendritic compartmentalization, and (6) field-conditional layer emergence.

The headline empirical contribution is the *pool-matched absorption* operator that the multi-axis substrate enables: cell-granularity composition of independently-trained organisms under a shared positional convention. Under v1.0, absorption required branch-granularity merging through a `MultiBranchOrganism` wrapper. Under v1.1, every donor cell is appended to the recipient at its corresponding pool, inheriting the recipient’s mask convention (LCN extension) and slotting into its pool structure. The composition preserves task-aware accuracy within $\Delta \leq 0.026$ on both donors and lifts full-softmax accuracy by +0.20 to +0.31.

We do not claim a new continual-learning benchmark result. The contribution is architectural: a substrate factored into orthogonal axes, plus a composition operator that the factoring enables. Not all axes improve the main benchmark when activated jointly—Axis 3 (depth growth) regresses accuracy and is now default-off—and we document these findings alongside the successes.

2 Related Work

Dendritic computation. Poirazi & Mel (2003) and Beniaguev et al. (2021) showed that a single biological neuron with active dendrites can implement functions comparable to a multi-layer perceptron. Axis 5’s compartmentalized branches are a deliberately lightweight instantiation of this idea: K fan-in partitions with per-branch nonlinearities and learned branch weights, sufficient to capture input-dependent gating without the full biophysical model.

Modular and mixture-of-experts architectures. Branch-Train-Merge (BTM; Li et al., 2022) trains expert LM branches independently and merges them at inference. Pool-matched absorption operates at a finer granularity—individual cells rather than entire branches—and preserves spatial locality constraints through the pool convention, but the motivation is the same: compose independently-trained specialists without joint retraining.

Locally-connected networks. Time-delay neural networks and CapsNets introduced locality as a structural prior. Triron’s LCN mask is opt-in locality on top of an otherwise flat-dense substrate: it constrains which upstream cells a given cell reads, but the substrate itself carries no hardwired spatial topology. This makes locality a tuneable axis rather than a fixed architecture choice.

Continual-learning substrates. PackNet (Mallya & Lazebnik, 2018), HAT (Serra et al., 2018), and DEN (Yoon et al., 2018) each protect learned structure against catastrophic forgetting through pruning, masking, or selective expansion. The v1 companion paper compared triron head-to-head with PackNet, HAT, Online EWC, and LwF; this paper does not re-run those benchmarks but inherits their conclusions as context for the architectural changes reported here.

3 Methods

3.1 The Triron Node, recapitulated

A v1.0 triron cell carries three scalars: a weight w (the learnable parameter), an EWC lock λ (the diagonal Fisher importance, anchoring w against drift on subsequent tasks), and a utility u (an exponential moving average of gradient magnitude, used by the pruner to identify dispensable cells).

Growth adds cells; pruning removes low- u cells; the frustration signal triggers growth when task loss stalls; dream-cycle consolidation replays manifold-sampled codes to re-anchor the head.

Version 1.1 extends each cell’s state with the following buffers, one per axis: **cell_position** $\in \mathbb{R}^3$ (developmental coordinate, used for pool assignment and LCN masks); **input_sources** $\in \mathbb{Z}^{\text{fan_in} \times 2}$ (Axis 1: source layer and node index for each fan-in column); **input_archived** $\in \{0, 1\}^{\text{fan_in}}$ (Axis 2: per-column archive flag); **axonal_gain** $\in \mathbb{R}^n$ (Axis 4: per-node output scalar); **branch_id**, **branch_weight**, **B_per_node** (Axis 5: dendritic partition and branch weights); **epi_A**, **epi_B** $\in \mathbb{R}^n$ (Axis 6: neighborhood-stress and self-activity field channels). Every buffer has a default that reproduces v1.0 behavior, so a default-constructed v1.1 cell is indistinguishable from a v1.0 cell in both the forward pass and the checkpoint format.

3.2 Axis 1: Long-range input reach

In v1.0, every cell reads only the immediately preceding layer’s output; cross-layer connections require manual wiring. Axis 1 replaces this implicit convention with an explicit **input_sources** tensor of shape $(\text{fan_in}, 2)$, recording the source layer index and node index for each fan-in column. A sentinel value of $(-1, -1)$ denotes the default sequential source (same column in the preceding layer), and the forward path’s fast branch detects the all-sentinel case and falls back to a plain **F.linear** call with zero overhead. When a non-sentinel entry is present, the forward path gathers the indicated activations before the matrix multiply, enabling skip connections and cross-layer reuse without network surgery. New connections are added via **set_input_source(net, layer, node, col, src_layer, src_node)**, which writes the sentinel pair and extends the LCN mask if active.

3.3 Axis 2: Plastic fanout sparsity

The v1.0 pruner marks a cell as **archived** (row-side), freezing its outgoing weights. Axis 2 adds the column-side mirror: an **input_archived** flag per fan-in column, indicating that the upstream source cell has been archived. When **archive_input_cascade** is enabled, archiving a cell propagates the flag to every downstream column that reads it, so source-side apoptosis is a single API call rather than a manual traversal. Archived columns are excluded from gradient updates but remain in the forward pass, preserving the learned representation while preventing further drift on the frozen source’s contribution.

3.4 Axis 3: Depth growth

The function **insert_layer(net, after_idx)** splices a new **TrioronLayer** between layers **after_idx** and **after_idx+1**. The inserted layer is identity-initialized in the style of Net2Net, with optional Gaussian noise, so it begins as a pass-through and the network’s input–output map is unchanged at the moment of insertion. A per-slot cap **K_insert** prevents unbounded depth growth at any single position.

Depth insertion is gated by the **allow_insert_layer** flag on the active **TrioronProfile**. It defaults to OFF in the OPEN, CLASSIFICATION, and EDGE profiles and is enabled only in REASONING, because pairwise ablation on the chained-15 benchmark showed that depth growth regresses accuracy when combined with Axes 4 and 5. The intended usage pattern is a one-shot depth injection at a curriculum boundary, not continuous insertion during training.

3.5 Axis 4: Slow axonal gain

Each node carries a per-node multiplicative scalar `axonal_gain` that scales its outgoing edge contributions, giving an effective weight $W_{\text{eff}}[i, :] = \text{axonal_gain}[i] \cdot W[i, :]$. The gain operates on a slower timescale than ordinary weight learning: it is not updated by backpropagation but is instead written by external signals—reward, attention, manual priors—via `set_axonal_gain(net, layer_idx, gains)`. EWC anchors the gain alongside W , so consolidated gain values resist drift on subsequent tasks.

The purpose is task-dependent up- or down-regulation of a cell’s influence without modifying the learned weights themselves, analogous to neuromodulatory gain control in cortical circuits.

3.6 Axis 5: Dendritic compartmentalization

Each cell can host $K \geq 1$ dendritic branches, controlled by the profile parameter `B_max`. A flat `branch_id` vector partitions the fan-in columns into K groups; each branch computes a local dot-product over its partition and passes the result through a per-branch nonlinearity σ_{branch} (default `quad`: $\sigma(x) = x + x^2$, an NMDA-style supralinear activation) before soma-level pooling via learned `branch_weight`. When $K = 1$ the forward pass is byte-identical to v1.0 (a plain `F.linear` call). New branches are added with `grow_branch`; `inherit_dendrite` seeds a child branch from a parent’s partition for sister-specialist differentiation.

Phase 6 falsification confirmed that dendritic compartmentalization is load-bearing: $K=2$ lifts $K=1$ by +0.49 absolute accuracy on a concentric-ring discrimination task and by +0.76 absolute on a four-class variant, with the trigger arm matching forced-branch performance within noise.

3.7 Axis 6: Field-conditional layer emergence

Two per-node field channels drive conditional cell emergence. `epi_A` tracks chronic neighborhood frustration: a Gaussian diffusion of each cell’s `internal_stress` over the spatial positions of nearby cells. `epi_B` tracks self-activity as an exponential moving average. The spawn rule `axis6_spawn` selects candidates by a relative criterion: cells whose neighbors are stressed (high `epi_A`) but that are themselves quiet (low `epi_B`) become spawn sites. New cells inherit position from their parent plus Gaussian jitter, so they appear near the high-stress region rather than at arbitrary locations.

After each task, cells whose engagement fraction exceeds a threshold earn “credit” and have their gradients zeroed on all future tasks—a credit-based row-freezing mechanism that is strictly stronger than EWC’s quadratic penalty, because frozen rows experience zero drift rather than penalized drift.

3.8 Spatial substrate: positions, pools, and locality

Every cell carries a 3D continuous coordinate `cell_position`, assigned during a developmental phase from the positions of upstream cells. A `pool_id` partitions the $[0, 1]^2$ plane into a grid (default $4 \times 4 = 16$ pools) used for pool-matched absorption (§3.9).

Locally-Connected masks (`enable_lcn`) optionally tie each cell’s weight row to a spatial window of input cells. The mask can be a soft Gaussian ($\exp(-d^2/2\sigma^2)$) or a hard top- K nearest-neighbor selection; it is applied element-wise to W during the forward pass so that off-window weights stay zero. When cells are added via `grow_node`, `extend_lcn_mask` automatically extends the mask to cover the new topology. LCN defaults to off, preserving the substrate’s natively flat-dense character; it is enabled per-layer or per-profile when spatial locality is beneficial.

3.9 Pool-matched absorption

The architectural unlock of v1.1 is cell-granularity composition. Under v1.0, absorption operated at branch granularity through a `MultiBranchOrganism` wrapper and required donors to share the L0 perception layer’s exact weights (the $R \cdot S$ handshake at L0). Under v1.1, every donor cell is appended to the recipient at its corresponding *pool*. Donor and recipient must share the upstream layer’s width (the $R \cdot S$ handshake) and the head’s class space; under those constraints, absorbed cells inherit the recipient’s mask convention (LCN extension) and slot into the recipient’s pool structure.

The composition proceeds as a four-mechanism stack:

1. `pool_matched_absorb`: transfer donor W rows, head columns, and biases into the recipient, bucketed by pool.
2. *Selective head merging*: zero out donor head columns for classes the donor never trained on, preventing spurious logit contributions.
3. `merge_manifold`: combine per-class (μ, σ) signatures from both donors into a single manifold archive.
4. `settle_head_via_manifold`: a 200-step head-only calibration pass under manifold replay, aligning the merged head without touching substrate weights.

First verification uses a chained-15 disjoint-class split (donor A on classes 0–3, donor B on classes 4–7, both with LCN on L1, soft $\sigma=0.25$, $k=8$). Task-aware accuracy is preserved within $\Delta \leq 0.026$ on both donors; full-softmax accuracy lifts +0.20 to +0.31; post-absorption locality reaches 0.78+ (vs. donor B solo 0.58), confirming that the merged substrate is *more* locality-clean than either donor alone.

4 Results

We report three categories of result: per-axis substrate self-tests (§4.1), the pool-matched absorption composition (§4.2), and backward compatibility with v1.0 donors (§4.3).

4.1 Substrate self-tests: per-axis end-to-end

Each axis was validated in isolation before joint evaluation.

Axis 5 (dendrites). Phase 6 falsification tests confirmed that dendritic compartmentalization is load-bearing. On a concentric-ring discrimination task, $K=2$ branches lifted $K=1$ by +0.49 absolute accuracy; on a four-class variant, the lift was +0.76 absolute. In both cases, the trigger arm (spawn a second branch when frustration fires) matched forced-branch performance within noise, validating the Phase 2.5 trigger machinery end-to-end.

Axis 6 (field-conditional emergence). The credit-assignment plus manifold-replay recipe on the chained-15 benchmark ($n=3$, 8 epochs) reached 0.959 ± 0.001 task-aware and 0.563 ± 0.004 full-softmax. Without manifold replay, credit alone reached 0.971 task-aware but only 0.168 full, confirming that manifold replay is load-bearing for full-softmax retention. Without credit (spawn only, no freezing), performance degraded to 0.735 task-aware and 0.073 full.

Axes 1–4 smoke tests. Axes 1 (input sources) and 2 (fanout sparsity) passed functional sentinels: cross-layer gather produces correct activations, and archive-input cascade propagates flags to all

downstream consumers. Axis 4 (axonal gain) correctly modulates effective weights under EWC anchoring.

Joint activation regression. When all six axes are activated simultaneously on the chained-15 benchmark ($n=3$), full-softmax accuracy regresses from the v1.0 baseline of 0.591 ± 0.002 to 0.413 ± 0.049 . Pairwise ablation identified Axis 3 (depth growth) as the primary culprit; it is now default-off in all profiles except REASONING. The credit+manifold+cosine recipe reported above uses Axis 6 alone and does not trigger this regression.

4.2 Pool-matched absorption: the LCN unlock

Table 1 summarizes the pool-matched absorption sentinel. Task-aware accuracy on both donor task slices is preserved within $\Delta \leq 0.026$; full-softmax accuracy lifts substantially because the merged head now covers both class sets under a single manifold archive. Post-absorption locality (0.78+) exceeds donor B’s solo locality (0.58), confirming that pool-bucketed absorption respects spatial structure.

Table 1: Pool-matched absorption on the chained-15 disjoint split (classes 0–3 / 4–7, LCN soft $\sigma=0.25$, $k=8$).

Condition	Task-aware	Full-softmax
Donor A solo (cls 0–3)	0.98	—
Donor B solo (cls 4–7)	0.98	—
A+B on A-tasks	≥ 0.95	+0.20 lift
A+B on B-tasks	≥ 0.96	+0.31 lift
Post-absorb locality	0.78+ (vs. B solo 0.58)	

4.3 Backward compatibility

Version 1.0 donor checkpoints load into the v1.1 substrate without modification. The `_load_from_state_dict` override detects missing v1.1 buffer keys (listed in `_TRIORON_2_0_KEYS`) and injects their defaults—e.g., `axonal_gain` is initialized to all-ones, `branch_id` to all-zeros (single branch), and field channels to zero. Removed buffers such as `head_provenance_mask` are silently discarded. At the profile level, setting `re_apply_after_donor_load=True` restores the active profile’s `branch_activation` post-load, so the regime choice (e.g., CLASSIFICATION vs. EDGE) survives donor absorption. This backward-compatibility path has been verified on all v1.0 checkpoints produced during the five-family competitor sweep.

5 Discussion

The axis decomposition converts the trioron substrate from a monolithic architecture into a composition surface. Each axis can be enabled, tuned, or disabled independently; profiles bundle axis settings into named presets (OPEN, CLASSIFICATION, EDGE, REASONING), so downstream users select a regime rather than configuring six axes individually. Pool-matched absorption—the mechanism the decomposition unlocks—is the first cell-granularity composition operator we have

for trained substrates: it merges independently-trained organisms without retraining, respecting spatial locality constraints through the pool convention.

The costs are real. The API surface is larger: six axis-specific write functions, a profile system, and the absorption stack. More critically, not all axis combinations improve performance. Axes 3, 4, and 5 activated jointly on the main benchmark regressed full-softmax accuracy by -0.178 absolute compared to v1.0, and Axis 3 is now default-off. The $n=3$ seed budget on the v1.1 credit+manifold recipe means that the 0.563 full-softmax result carries wider confidence intervals than the v1.0 headline (0.601 at $n=10$); the two are not directly comparable in statistical strength.

Looking ahead, the substrate remains natively flat-dense: locality is opt-in via LCN masks, and spatial position is a soft coordinate rather than a hardwired topology. This limits the substrate’s ability to exploit spatial structure in perceptual inputs (images, spectrograms) without an external perception stage. The next architectural question is whether the substrate can grow its own spatial priors through the existing axes, or whether a dedicated perception lift is required.

6 Conclusion

Triron 1.1 factors the monolithic v1.0 node into six orthogonal axes, each opt-in and each with its own write API and plasticity gate. The decomposition enables pool-matched absorption, a cell-granularity composition operator for independently-trained substrates that preserves task-aware accuracy within $\Delta \leq 0.026$ and lifts full-softmax accuracy by up to $+0.31$. Not every axis improves the main benchmark—Axis 3 regresses and is default-off—but the factoring itself is the contribution: a substrate that can be configured, composed, and extended at cell granularity.

Taken together, the six axes demonstrate that a single substrate carries the minimum capacity to form multiple adaptive layer types: dendritic compartmentalization (Axis 5) recovers transformer-like multi-head attention through learned branch partitions; locally-connected masks plus positional pooling approximate convolutional receptive fields without hard-coded kernels; depth growth (Axis 3) adds layers on demand rather than at design time; and field-conditional emergence (Axis 6) produces mixture-of-experts-style routing through credit-based specialization. None of these emerges as a pre-wired module—each arises from the same flat-dense cell under different axis configurations. The substrate does not yet match purpose-built architectures on any single axis, but it provides a unified, composable foundation from which transformer, convolution, depth, and routing behaviors can all be instantiated and freely combined within a single growing network. Full equations and pseudocode are in Appendix A.

A Detailed Methods

This appendix provides the full equations and pseudocode for the mechanisms summarized in §3. The canonical reference is the `triron_2_0.md` specification in the repository; the material below is a paper-formatted extract of the load-bearing formulas.

A.1 Locally-Connected mask construction

`build_lcn_mask` constructs a binary or soft mask of shape $(n_{\text{out}}, n_{\text{in}})$ from cell positions. In *soft* mode, each entry is $M_{ij} = \exp(-d_{ij}^2/2\sigma^2)$ where d_{ij} is the Euclidean distance between cell j (input) and cell i (output) in the $[0, 1]^2$ plane. In *hard* mode, each output cell retains its k nearest input cells: $M_{ij} = 1$ if j is among the top- k nearest neighbors of i , and 0 otherwise. The mask is applied element-wise during the forward pass: $W_{\text{eff}} = M \odot W$.

`extend_lcn_mask` follows a two-phase contract: (1) on `grow_node`, the mask tensor is resized to the new topology with the new rows/columns initialized to ones (permissive); (2) after positions are assigned to the new cells, the mask is recomputed for the affected rows/columns using the soft or hard formula above. This ensures that freshly grown cells begin with a valid locality window rather than inheriting stale distances.

A.2 Pool partition

The $[0, 1]^2$ position plane is partitioned into a $G \times G$ grid of pools (default $G=4$, giving 16 pools). A cell at position (x, y) is assigned pool identifier

$$\text{pool_id}(x, y) = \lfloor x \cdot G \rfloor \cdot G + \lfloor y \cdot G \rfloor,$$

where coordinates are clamped to $[0, 1-\epsilon]$ before flooring to avoid boundary overflow. The inverse, `pool_centroid`, maps a pool id back to the center of its grid cell:

$$\text{pool_centroid}(p) = \left(\frac{\lfloor p/G \rfloor + 0.5}{G}, \frac{(p \bmod G) + 0.5}{G} \right).$$

Pool-matched absorption uses `pool_id` to bucket donor cells into the correct recipient region, and `pool_centroid` to initialize positions for newly absorbed cells when the donor’s exact coordinate is not available.

A.3 Pool-matched absorption procedure

The absorption procedure is:

1. For each donor cell c in layer ℓ , compute $p = \text{pool_id}(c.\text{position})$.
2. In the recipient’s layer ℓ , find the pool- p bucket. If the bucket is at capacity (`max_per_pool`), skip c .
3. Append c ’s weight row W_c to the recipient’s weight matrix. Copy c ’s bias, λ , u , and all v1.1 buffers (`axonal_gain`, `branch_id`, etc.).
4. Extend the recipient’s head: copy the donor’s head column for cell c into the corresponding position in the recipient head’s weight matrix.
5. If LCN is active, call `extend_lcn_mask` to resize and recompute the mask for the new cell.
6. Zero out the donor’s head columns for classes the donor never trained on (selective head merging).

After all cells are absorbed:

7. Call `merge_manifold` to combine per-class (μ, σ) signatures from both donors.
8. Run `settle_head_via_manifold` for 200 steps of head-only gradient descent under manifold-sampled synthetic codes, aligning the merged head without touching substrate weights.

A.4 Manifold archive

The manifold archive stores, for each class y , the mean μ_y and standard deviation σ_y of L0 output codes observed during training, both tensors of shape $(d_{L0},)$. Synthetic codes are sampled as

$$z \sim \mathcal{N}(\mu_y, \text{diag}(\sigma_y^2)), \quad z \leftarrow \max(z, 0),$$

where the clamp at zero enforces the ReLU contract of L0’s output activation, preventing the head from seeing codes outside the representable range.

`settle_head_via_manifold` runs 200 steps of Adam ($\text{lr} = 10^{-3}$) on the head’s weight and bias only, with all substrate parameters frozen. The loss is masked cross-entropy: for each mini-batch of synthetic codes, the loss is computed only over classes present in the manifold archive, so that classes from a donor that has not yet been absorbed do not contribute spurious gradients. This procedure is the final step of the absorption stack and is also used standalone after dream-cycle consolidation in v1.0.

B Reproducibility

All experiments use PyTorch with deterministic mode enabled (`torch.use_deterministic_algorithms(True)`) and explicit seed setting via `torch.manual_seed(seed)`. The chained-15 benchmark is invoked as:

```
TRIORON_MANIFOLD_REPLAY=1 python3 bench/bench_chained_15task.py
```

The environment variable `TRIORON_MANIFOLD_REPLAY=1` is required for standalone benchmark runs; the API’s `build_donor` sets it automatically.

Four profile presets configure axis defaults:

- OPEN: all axes at v1.0 defaults (no axes active).
- CLASSIFICATION: Axis 6 credit + manifold; LCN off; $B_{\max}=1$.
- EDGE: same as CLASSIFICATION with int8 dream archive enabled.
- REASONING: Axis 3 (depth growth) enabled; $B_{\max}=2$; intended for complex-task curricula.

Multi-seed runs use seeds $\{0, 1, \dots, n-1\}$. The absorption sentinel is `tests/pool_matched_absorb_lcn_test.py`; per-axis smoke tests are in `tests/test_node.py`. Source code is available at <https://github.com/Marcrochat/trioron-project>.

References

Marcelinus R. Hatorangan. Trioron: a dynamically growing neural architecture for resource-bounded continual learning. Unpublished manuscript. Repository: <https://github.com/Marcrochat/trioron-project>, 2026.